

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Soundariya M	Reg. No.:	192524342
Section / Batch:	Btech-AI&DS	Date:	21:08:2026

Instructions to Students

- Answer both questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Description	Questions	Total Marks
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1 – Q2	100

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

SECTION A – Coding Challenge Task

Question 1:



SIMATS Online Programming Lab
DAA PROGRAMMING

Soundariya M
1925248425A

CO4-AT1-Coding Challenge

Language: Python C++ Java

Q1
Dynamic Programming
50 marks

Q2
Dynamic Programming
50 marks

2/2 answered

Q1: Dynamic Programming

50 marks

House Robber Problem

A robber plans to steal money from houses arranged in a line, but cannot rob two adjacent houses. Each house has a certain amount of money. The goal is to maximize the amount stolen without triggering alarms. Use Dynamic Programming to determine the optimal strategy.

Input Format:
n = number of houses
arr[] = money values
Sample Input:
n = 4
arr = 2 7 9 3
Expected Output:
Max Loot = 11

Your Code

```
1 n = int(input())
2 arr = list(map(int, input().split()))
3
4 if n == 0:
5     print("Max Loot = 0")
6 elif n == 1:
7     print("Max Loot =", arr[0])
8 else:
9     prev2 = arr[0]
10    prev1 = max(arr[0], arr[1])
11
12    for i in range(2, n):
13        current = max(prev1, prev2 + arr[i])
14        prev2 = prev1
15        prev1 = current
16
17    print("Max Loot =", prev1)
```

Submitted

Input

4
2 7 9 3

Output

Max Loot = 11

Question 2:



SIMATS Online Programming Lab
DAA PROGRAMMING

Soundariya M
1925248425A

CO4-AT1-Coding Challenge

Language: Python C++ Java

Q1
Dynamic Programming
50 marks

Q2
Dynamic Programming
50 marks

2/2 answered

Q2: Dynamic Programming

50 marks

Minimum Number of Coins (Unbounded Knapsack)

Given coin denominations and a target value, determine the minimum number of coins required where each coin can be used multiple times. The objective is to reach the exact amount with minimum coins. Use Dynamic Programming to compute the solution efficiently.

Input Format:
n = number of coins
coins[] = denominations
amount = target
Sample Input:
n = 3
coins = 1 3 4
amount = 6
Expected Output:
Min Coins = 2

Your Code

```
1 n = int(input())
2 coins = list(map(int, input().split()))
3 amount = int(input())
4
5 # dp array to store minimum coins for each amount from 0 to target
6 dp = [float('inf')] * (amount + 1)
7 dp[0] = 0
8
9 for i in range(1, amount + 1):
10     for coin in coins:
11         if coin <= i and dp[i - coin] != float('inf'):
12             dp[i] = min(dp[i], dp[i - coin] + 1)
13
14 print("Min Coins =", dp[amount])
```

Submitted

Input

3
1 3 4
6

Output

Min Coins = 2

Code of Conduct

I Soundariya M(192524342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Soundariya M _____

Mark Evaluation:

Questions	Problem Understanding (20)	Algorithm (25)	Code Implementation (20)	Competitive Performance (20)	Test Case Handling (15)	Total Marks (Each - 50)
Q1						
Q2						
Overall Total						

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____